WhatsApp CRM - SaaS Launch & Feature Development

Developer-ready backlog | Version 1.1 | 25 October 2026 | Supersedes version 1.0 of the same day

Goal: convert the existing multi-number, multi-agent WhatsApp CRM into a paid multi-tenant SaaS product, then improve contact management, direct messaging, tagging/filtering, and team response analytics.

What changed in 1.1

- **Every task now has a status, checked against the code and the automated tests**, not against screenshots. Version 1.0 said the screens were "not backend-verified"; this version replaces that with what was read in the repositories.
- **Several tasks were already partly or fully built** (subscription periods and renewals, the AI assistant, follow-ups, contact rename, start-chat from Contacts). Their remaining scope is smaller than the 1.0 effort suggests.
- **CRM-002 is rewritten for Razorpay** (orders, verified checkout, signed webhooks, idempotency, reconciliation). Recurring autopay is split out as **CRM-016** and GST invoices as **CRM-017**.
- **Decisions the owner must make before CRM-002 starts** are listed at the end of this document (annual price, UAE/AED, GST, autopay, who may pay).

Status key: **Done** built, tested and live **Partly** some of the scope is built **Not started** **New** added in 1.1.

Product today (read from the code, 25 October 2026)

- **Inbox and numbers:** shared live inbox over several linked WhatsApp numbers; staff see their own number's chats and chats assigned to them, owners see all, enforced on the server; assign, resolve, private notes with @mentions; saved replies; alerts when a phone goes offline.
- **Plans:** Free, Starter, Growth, Business, Enterprise are enforced on the server (numbers, people on Free, broadcasts, automations, sales pipeline, audit log, API, AI). The platform admin chooses a plan when creating a company and can change it.
- **Subscriptions (new since 1.0 was drafted):** each company has a billing cycle, a period and a renewal date; the platform admin records each payment; a smaller plan or Free can be scheduled for the end of the period and is applied by a daily job; owners are reminded once before renewal and once when overdue; nothing is switched off automatically for non-payment.
- **Sales pipeline:** board, list and Today list, follow-ups with reminders, a bar in every chat, lead rules, Growth plan and above.
- **Broadcasts and automations:** three-step broadcast wizard with consent checks, ten ready-made messages, 14 ready-made rules, keyword / first-message / after-hours triggers.
- **Riwo (AI assistant):** reply drafts, improve, translate, chat summaries; company switch, daily allowance, usage shown in Your plan and on the platform list.
- **Not present:** any online payment, invoices, reply-time analytics, colour labels, Inbox filters beyond All / Unread / Groups, round-robin assignment.

Implementation order / delivery gates

Gate	Must ship	Exit condition	Status 25 Oct
G1 / P0	Plans + paid checkout + subscriptions + usage enforcement	Real test payment correctly creates/renews one tenant subscription; no cross-tenant access.	Partly Subscriptions, renewals and limits are built. Paid checkout (CRM-002) is not.
G2 / P0	Contact names + direct chat + labels and filters	Agent can create/edit/contact, initiate allowed outbound messaging and find contacts by label.	Partly Rename on Contacts and Message from Contacts exist; labels and filters do not.
G3 / P1	Response-time reporting + inbox productivity	Owner sees accurate agent stats including unanswered chats and SLA performance.	Not started
G4 / P1-P2	Renewal UX + reports + automation + optional AI	Production readiness sign-off and pilot customer acceptance.	Partly Renewal reminders, AI and automation are built; reports are not.

Master task register

Task ID	Priority	Scope	Status	Remaining (indicative effort)
CRM-001	P0	Plan catalog & pricing	Partly	Prices and plans into the database, yearly price, trial policy, Plans & Billing menu (1.5-2d)
CRM-002	P0	Checkout, gateway & verified webhook (Razorpay)	Not started	Orders checkout, signed webhook, idempotency, reconciliation (4-6d)
CRM-003	P0	Subscription lifecycle & access limits	Partly	Trial, grace, expired, pending payment; limits on contacts and broadcasts; usage counters (2-3d)
CRM-004	P0	Contact name save/edit	Partly	Edit from the Inbox panel, separate CRM name from WhatsApp name, change audit (0.5d)
CRM-005	P0	Direct message / start chat	Partly	New chat in Inbox with a typed number, sender choice, number validation (1d)
CRM-006	P0	Labels, contact & inbox filters	Partly	Coloured, company-managed labels; bulk tagging; label filters in Contacts and Inbox (1.5-2d)
CRM-007	P1	Reply-time metrics and leaderboard	Not started	3-5d
CRM-008	P1	Inbox filters + quick actions	Partly	Mine / team / unassigned / labelled / overdue filters, pinned, reply-collision warning (1-1.5d)
CRM-009	P1	Follow-up and escalation	Partly	Tasks without a lead, escalation of overdue tasks (1d)

Task ID	Priority	Scope	Status	Remaining (indicative effort)
CRM-010	P1	Billing self-service + invoices	Partly	Invoices, payment history, pay / renew / cancel / retry, finance role (2-3d, after CRM-002)
CRM-011	P1	Owner daily reporting	Not started	2-3d, after CRM-007
CRM-012	P1	Operational observability / number health	Partly	Heartbeat, failed sends per number, queue per number (1-2d)
CRM-013	P2	Lead assignment rules	Not started	2-4d
CRM-014	P2	AI chat summaries (opt-in)	Done	Sentiment is not included (0.5d if wanted)
CRM-015	P2	SaaS super-admin console	Partly	Platform usage totals, per-company number health, billing audit view (2-3d)
CRM-016	P1	Razorpay autopay (recurring subscriptions)	New	3-4d, after CRM-002 and the owner decisions at the end
CRM-017	P1	GST invoices and receipts	New	2-3d, with CRM-010

Estimates are illustrative developer-days for what remains, not committed timelines. The 1.0 efforts were for the whole task; the sales pipeline, broadcast wizard, automation rules and AI assistant were built outside this list.

Task detail

CRM-001 | SaaS plan catalog (P0) Partly

Built: five plans enforced on the server (`plans.js`): number limit, people (Free is one person), and which features each includes. The platform admin picks a plan when creating a company and can move it later; owners see their plan, what it includes and what it does not. Prices are shown in AED and INR on the public pricing page.

Remaining: prices exist only on the website, so checkout cannot trust them. Move plan code, name, interval, `price_minor`, currency, tax mode, limits and feature flags into a `plans` table that the server reads and the platform admin edits without a deploy. Add the yearly price (the pricing page says "monthly or annual" but shows only monthly), the trial policy, and a Plans & Billing menu for owners.

Acceptance (unchanged): owner can compare plans; admins can change plans without deploying code; no frontend price manipulation; limit and billing interval shown clearly. Dependencies: CRM-002.

CRM-003 | Subscription lifecycle + limits (P0) Partly

Built: the state of a company is worked out from its dates each time: free, legacy, no payment recorded, active, due soon, overdue, changing, cancelling. Period start and renewal date, a scheduled downgrade or cancellation applied by a daily job, in-app reminders, number / people / feature limits enforced in the API (not the screen only). A smaller plan removes nothing the company already has.

Remaining: trial, grace and expired states; a pending state while a payment is in progress; limits on contacts and broadcasts (`max_contacts`, monthly broadcast count) with soft-limit warnings; usage counters (`tenant_usage_counters`). Overdue deliberately restricts nothing today: whether and when it should (grace days, then paid features off) is one of the owner decisions at the end of this document.

Acceptance: trial / purchase / upgrade / expiry flows tested; tenant cannot exceed hard limits; existing records remain recoverable after expiry. Dependencies: CRM-001, CRM-002.

CRM-004 | Save/edit contact name (P0) Partly

Built: a contact's name is edited on the Contacts page (`PATCH /api/contacts/:id`) and shows in the Inbox, Contacts and Pipeline.

Remaining: edit from the Inbox right panel; keep the WhatsApp profile name apart from the CRM name (`contacts.name` is one field today); normalise numbers to E.164 where possible; write an audit entry for each change; define duplicate handling (a number is unique per company today).

Acceptance: rename reflects in Inbox, Contacts and Pipeline without losing history; duplicate contact handling specified.

CRM-005 | Direct WhatsApp message (P0) Partly

Built: "Message" on a contact opens or creates the chat on a connected number (`POST /api/conversations`); sending uses the normal send path with rate limits, delivery states and sender identity.

Remaining: New chat in the Inbox for a typed number (validate format, choose the sending number from the ones the agent may use); show a warning when the contact has not agreed to messages. Numbers are linked like WhatsApp Web, so approved-template rules do not apply here; they would only apply if an official API connection is ever added.

Acceptance: send only when channel rules allow; message appears in the matching thread; failure has an actionable state. Dependencies: CRM-004.

CRM-006 | Labels & filtering (P0) Partly

Built: free-text tags on a contact, shown in the Inbox panel and used to pick a broadcast audience.

Remaining: company-owned, coloured labels (`labels` and `contact_labels` , unique per company); many labels per contact; bulk tagging after selection; label filter in Contacts and Inbox; filter combinations (label, assigned agent, number, unread, last activity). Move existing tags into labels without losing any.

Acceptance: add/remove shows at once; filtering works with several labels; labels never leak between companies. Dependencies: CRM-004.

CRM-007 | Employee reply analytics (P1) Not started

Unchanged from 1.0. Inputs exist (message timestamps, assignment, who sent each reply, notes kept apart from replies); nothing computes first or average response time yet. Compute from the customer's message to the next human reply, exclude automatic replies and team notes, report median and p90 with a minimum sample, and let the owner filter by date, team and number and export CSV.

CRM-008 | Inbox quick actions (P1) Partly

Built: All / Unread / Groups tabs, search, switching between numbers, saved replies, assign and resolve, private notes.

Remaining: Mine / team / unassigned / labelled / overdue filters on the server with paging, pinned or priority chats, a warning when two people type a reply at once. Dependencies: CRM-006.

CRM-009 | Follow-up reminders (P1) Partly

Built: follow-ups on a lead with a due time, owner, notes and a done state; a Today list; one reminder when due (moving the time reminds again); overdue shown on the board, list and in the chat bar.

Remaining: a task from a conversation that has no lead; escalate an overdue task to the team lead or owner; an audit entry for reassignment.

CRM-010 | Customer billing self-service (P1) Partly

Built: "Your plan" shows plan, billing cycle, renewal date and state, numbers used, and Riwo use today and this month, for owners and administrators only.

Remaining: invoices and payment history for the owner, pay now / renew / upgrade / cancel / retry a failed payment, a Finance role (today only owner and admin; a "billing" label exists but gives no permission). Proration and downgrade policy configurable. Dependencies: CRM-001 to CRM-003, CRM-017.

CRM-011 | Owner daily activity report (P1) Not started

Unchanged. The Overview screen shows basic counts today; the number / team / date-wise report with drill-down, agent scorecards and export is still to build. Dependencies: CRM-007, CRM-009.

CRM-012 | WhatsApp number health (P1) Partly

Built: connected / disconnected / failed status with the reason, an alert to the owner when a phone drops, queue status on the System screen, a read-only server report (`scripts/diagnose-numbers.mjs`) and a log line for every pairing step.

Remaining: last heartbeat, reconnect state, failed sends and queued messages per number on screen; alerts on persistent errors.

CRM-013 | Automatic lead assignment (P2) Not started

Unchanged. A rule can assign a chat to a named teammate; round-robin, team ownership, workload routing and working hours are not built. Dependencies: CRM-008.

CRM-014 | AI conversation summaries (P2) Done

Built: Riwo summarises a chat, drafts and improves replies and translates; a person always reads and sends. Companies can switch it off, each has a daily allowance, usage is metered and shown, customer messages are treated as data (prompt-injection tests), phone numbers and media are not sent, and nothing is sent automatically. **Not included:** sentiment.

CRM-015 | SaaS super-admin console (P2) Partly

Built: company directory with plan, number limit, people, live numbers, subscription state with overdue / renewing / unrecorded filters, record a payment, schedule a change, suspend and reactivate, AI use and allowance, an audit entry for each change. The platform admin cannot read a company's chats.

Remaining: platform-wide usage totals, number health per company, a billing audit view, support access that is logged.

CRM-002 | Razorpay checkout + webhook (P0) Not started

Build the first paid path with Razorpay **Orders + Standard Checkout**: one payment pays for one period (a month or a year), which is exactly what the subscription record already models. It reuses the existing `recordPayment()` so a Razorpay payment and a payment recorded by hand end up in the same place. Autopay is CRM-016.

Flow

1. The owner opens Plans & Billing, picks a plan and monthly or yearly, and gives billing details if none are saved (CRM-017).
2. `POST /api/billing/checkout` (owner only): the server finds the company from the login, reads the price from the plans table, adds tax, writes a `payment_orders` row (status *created*) and creates a Razorpay Order: amount in paise, currency, `receipt` = our order id, `notes` = company id, plan, cycle. It returns only the public key id and the order id.
3. The browser opens Razorpay Checkout with that order. Card and bank details never touch Scriwo.
4. On success the browser posts `razorpay_order_id`, `razorpay_payment_id`, `razorpay_signature` to `POST /api/billing/verify`. The server recomputes `HMAC-SHA256(order_id + "|" + payment_id, key_secret)` using the order id **from its own record, not from the browser**, and compares in constant time. A valid signature shows "payment received, activating" but is not by itself the activation.
5. **Activation** happens in one function, `settleOrder()`, called by the signed webhook (`payment.captured` / `order.paid`) or by the reconciliation job. It locks the order row, checks status, amount, currency and company, then calls `recordPayment()` and moves the order to *paid*. Whichever runs first wins; the second finds the order already paid and does nothing.
6. The owner and administrators get an in-app "payment received" notice; an invoice is issued (CRM-017).

Webhook rules

Rule	Requirement
Endpoint	`POST /api/billing/razorpay/webhook`, public, mounted **before** the global JSON parser with `express.raw`, because the signature covers the exact bytes received.
Signature	`X-Razorpay-Signature` = HMAC-SHA256 of the raw body with the **webhook secret** (not the API key secret), compared in constant time. Invalid: 400, nothing stored except a log line without the body.
Idempotency	The event id (`X-Razorpay-Event-Id`) is inserted into `gateway_webhook_receipts` with a unique constraint first; a duplicate returns 200 and does nothing. Events can arrive more than once and out of order, so every handler checks the order's current state.
Replay safety	Unique event id plus the order state check; a replayed old event cannot extend a period twice.
Checks before activating	Order exists and belongs to the company in its own notes; payment amount and currency equal the order's; payment status is captured. Any mismatch marks the order *mismatch*, activates nothing and raises an alert for the platform admin.
Failed / pending	`payment.failed` records the reason and leaves the company unchanged; the screen offers retry with a new order.
Refunds	A refund event is stored and shown to the platform admin; it does not remove access by itself (P1).

Rule	Requirement
Response	200 within a few seconds after the receipt is stored; heavy work after responding or by the job.
Audit	Every order, receipt and settlement writes an audit entry; secrets and full card data are never logged.

Reconciliation and recovery

A job every few minutes looks at orders still *created* or *attempted* after 10 minutes and asks Razorpay for the order's payments; a captured payment is settled the same way a webhook would. This covers lost webhooks and a customer who closed the tab. Orders with no payment expire after 24 hours.

Configuration (server only)

`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET` in the server's environment file, never in the repository or the browser. The mode (test or live) follows the key prefix `rzp_test_` / `rzp_live_`; `/health` reports whether payments are configured and in which mode. Checkout is switched off when any of the three is missing.

Acceptance

- Successful verified payment activates exactly one company, once; a duplicate webhook creates no duplicate payment, period or invoice.
- A failed or pending payment does not activate; a payment for the wrong amount, currency or company does not activate.
- The company cannot be chosen by the browser: another company's order id returns 404.
- Activation works when only the webhook arrives, when only the browser verify arrives, and when both arrive together.
- Paying early never shortens a period; paying for a period already paid up shows a warning first.

Dependencies: CRM-001. Verified from public documentation: webhook signature, checkout signature and the `X-Razorpay-Event-Id` header (confirm the header in the test-mode delivery log). Everything else is to be proven in Razorpay test mode before live keys are used.

CRM-016 | Razorpay autopay (P1) New

Recurring charge through Razorpay Subscriptions: a Razorpay plan per Scriwo plan and cycle, a first authenticated payment that registers the mandate (card, UPI autopay or e-mandate), then `subscription.charged` events that each call `recordPayment()`. Map `halted` to payment overdue and `cancelled` to a scheduled end of period; let the owner cancel autopay; keep the manual "pay now" path.

Why it is separate: amounts are fixed on the Razorpay plan, so a price change needs a new plan; and with 18% GST the Business monthly amount (about 17,699 rupees) is above the 15,000-rupee level up to which recurring debits are usually processed without the customer approving each one. Confirm the current limits with Razorpay before building. Yearly plans are not suited to autopay.

Acceptance: one charge, one period, however many times Razorpay repeats the event; a failed charge shows overdue and the retry state; cancelling stops the next charge and keeps the paid period.

CRM-017 | GST invoices and receipts (P1) New

Billing details per company (legal name, address, state, GSTIN if any), an invoice number series, tax shown separately (18% GST for India), a PDF the owner can download, and a receipt for every settled payment. The invoice must match the paid transaction exactly; manual payments get an invoice the same way. The accountant confirms the invoice fields and service code before the first live invoice.

Acceptance: downloadable invoice matches the paid transaction; numbers never repeat or skip silently; only owner and finance roles can see invoices.

Required database / API contract

Entity	Status	Notes
Company plan, number limit, status	Exists	`tenants`: plan name, `sessions_limit`, status, AI allowance.
Subscription period	Exists	`tenants`: `billing_cycle`, `period_start`, `period_end`, `scheduled_plan`, reminder markers (migration 017).
Payments recorded	Exists	`subscription_payments`: plan, cycle, period, amount, currency, reference, who recorded it.
Audit	Exists	`audit_events`; subscription and plan changes write to it.
Plan catalogue with prices	To add	`plans` and entitlements (CRM-001): code, interval, `price_minor`, currency, tax mode, limits, features, sort order.
Orders and payments	To add	`payment_orders` (company, plan, cycle, amount, tax, currency, gateway order id, status, expiry) and `payments` (gateway payment id unique, status, method, raw reference, timestamps).
Webhook receipts	To add	`gateway_webhook_receipts`: event id unique, event, payload hash, received, processed, result.
Subscription events	To add	`subscription_events`: a readable history (created, renewed, changed, cancelled, overdue).
Invoices, billing details	To add	`invoices`, `billing_profiles` (CRM-017).
Usage counters	To add	`tenant_usage_counters` (CRM-003).
Labels	To add	`labels`, `contact_labels` (CRM-006); tags move into labels.
Response measurements	To add	Event stream or table for CRM-007 and a per-team SLA setting.
Endpoints	Partly	Exist: `/api/admin/companies/:id/subscription/*`, `/api/contacts/:id`, `/api/conversations`, `/api/leads`. To add: `/api/plans`, `/api/billing/checkout`, `/verify`, `/subscription`, `/invoices`, `/api/billing/razorpay/webhook`, `/api/labels`, `/api/reports/team-response`.
Roles	Partly	Owner, Admin, Agent exist. Team Lead and Finance do not; "billing" is a label on a person, not a permission.

Mandatory release acceptance / QA checklist

Check	Status
No chat history, contacts, numbers, pipeline cards or automation rules lost during migration (migrations are additive; a smaller plan removes nothing)	**Covered** migration and plan tests
Production checkout uses an authenticated company, verified server-side events, idempotent processing and tax-compliant receipts	**Open** CRM-002, CRM-017
Limit checks cannot be bypassed by direct API calls; upgrades and expiry stay consistent when a webhook is late	**Partly** numbers, people and features enforced; late-webhook case with CRM-002
Cross-tenant tests for contacts, chats, users, numbers, exports, billing, reports and webhooks	**Partly** contacts, chats, numbers, leads covered; billing and reports with their tasks
Owner overview vs agent-only access; employee performance excludes team notes and automatic messages	**Partly** access covered; performance with CRM-007
Message sending follows the actual WhatsApp connection method, consent requirements and platform rules	**Partly** broadcasts need permission and use delays; direct-message rules with CRM-005
Observability: logs, failed jobs, payment reconciliation, backup/restore, rollback migration, staging smoke tests	**Partly** deploy health check and rollback exist; reconciliation and a staging smoke test are new work
Razorpay (new): test-mode run covering a good payment, a failed payment, a tampered signature, a wrong amount, a wrong company, a duplicate webhook, a webhook before the browser verify, the browser verify before the webhook, a lost webhook recovered by reconciliation	**Open** CRM-002
Razorpay (new): webhook delivery from Razorpay's servers to the production address confirmed in the dashboard's delivery log; live keys only after the test run is signed off	**Open** CRM-002
Sign-off from the product owner: sample checkout, one direct message, contact rename, label filter and 5 manually verified reply-time records	**Open**

Decisions needed from the owner before CRM-002

#	Decision	Why it matters	Recommendation
1	Yearly price	The pricing page and the terms offer annual billing but show no yearly price. The checkout needs one number per plan.	Set it per plan, for example twelve months for the price of ten, and publish it.
2	UAE customers and AED	Razorpay takes AED from international cards only if International Payments is activated on the account, and settles in rupees. Without it, UAE customers can only be invoiced and recorded by hand.	Ask Razorpay to activate it; until then keep AED companies on invoice.

#	Decision	Why it matters	Recommendation
3	GST invoicing	Indian prices exclude 18% GST. The order must include tax and an invoice must follow.	Give the GSTIN, invoice series and service code from your accountant; build CRM-017 with CRM-002.
4	Autopay now or later	Autopay adds setup, mandate limits and a second set of events.	Later (CRM-016). Launch with "pay for the next period", which matches the current terms.
5	Trial and late payment	A trial needs a start and end; overdue today restricts nothing.	Free plan stays the trial. Decide the grace days after which paid features switch off.
6	Who may pay and see invoices	Owner only, or also an administrator or a new Finance role.	Owner first; add Finance with CRM-010.
7	Razorpay account	Test keys and a webhook secret are needed to build; live KYC and activation of the methods to go live.	Create test keys and the webhook now; put them in the server's environment file, never in chat or in the repository.

Dev workflow / definition of done (unchanged)

For every task: (1) inspect existing implementation, (2) agree schema + permissions, (3) implement migration/API/UI, (4) add unit/integration tests, (5) validate in staging against actual WhatsApp channel constraints, (6) attach screenshots/test cases to PR, (7) deploy with backup and rollback plan, (8) mark task Done only after owner acceptance.

Version history

Version	Date	Change
1.0	25 October 2026	First developer backlog: 15 tasks, gates, contract and QA checklist.
1.1	25 October 2026	Status of every task checked against the code; Razorpay specification for CRM-002; CRM-016 (autopay) and CRM-017 (GST invoices) added; contract and QA checklist updated; owner decisions listed.